Writeup

The seeded generator (seed=42) produces **3 fictional apps** — `ledgerly` (finance), `promptly` (AI assistant), `streamio` (video) — over a **30-day window**, ~400 users per app, covering the full lifecycle (install → engage → pay → churn) plus injected duplicates, malformed payloads, and traffic spikes.

1. Event format — one envelope every app shares

Every app sends the same top-level envelope; each event type carries its own payload inside a flexible `properties` JSON field.

Field	Type	Why
`event_id`	STRING	unique id → enables deduplication
`app`	STRING	which app sent it
`event_name`	STRING	e.g. install, purchase, feature_used, llm_call
`event_type`	STRING	financial / engagement / backend (coarse grouping)
`user_id`	STRING	the user
`event_time`	TIMESTAMP	when it happened (client clock, epoch ms → TIMESTAMP)
`ingested_at`	TIMESTAMP	stamped on receipt; `ingested_at − event_time` = pipeline lag
`version`	STRING	app version (varies over time in the data)
`region`	STRING	US / EU / APAC / LATAM
`properties`	JSON	the per-event payload (price, feature, tokens, …)

The fixed columns are what every event has in common; `properties` carries what differs. A purchase puts `{product_id, price, currency}` there; an `llm_call` puts `{model, provider, input_tokens, output_tokens}`; a `feature_used` puts `{feature}`.

A new app or event type joins with no schema change and no pipeline redeploy — it just sends new keys inside `properties`. The ingestion service validates only the fixed required fields, so it doesn't need to know the new shape. **What that flexibility costs at query time:** reading a `properties` field needs `JSON_VALUE(properties,'$.x')` plus a `CAST`, and those fields are not typed or validated by the warehouse (a bad value only surfaces at read time). That is the deliberate trade — schema flexibility now, a little query-time work and weaker type guarantees later.

2. Table layout — designed around how it's queried

Single landing table `analytics.events` with the 10 envelope columns above:

```
CREATE TABLE analytics.events ( … 10 columns …, properties JSON )
PARTITION BY DATE(event_time)
CLUSTER BY app, event_name;
```

- **`PARTITION BY DATE(event_time)`** — time-range questions prune to the relevant day partitions instead of scanning all history (~30 partitions for this dataset).
- **`CLUSTER BY app, event_name`** — within a partition, rows are grouped by app then event, so **a question about one app does not scan the other apps' data**.
- **`properties` as JSON** — the flexibility above, at the cost of `JSON_VALUE` + `CAST` on read.

This is where ingestion decides query cost: the same partition/cluster layout is what lets a one-app, few-day question read a tiny slice rather than the whole table.

Duplicates and malformed data are handled by design:

- **Duplicates** — Pub/Sub is at-least-once and clients retry, so duplicate `event_id`s are allowed to LAND (BigQuery doesn't enforce uniqueness on insert), and are removed at QUERY time with `ROW_NUMBER() OVER (PARTITION BY event_id ORDER BY ingested_at)`. So a purchase that arrived twice is counted once. The protection lives in the query/serving layer, keyed on `event_id`.
- **Malformed** — invalid JSON and events missing a required field are rejected at the ingestion edge (400, logged, countable). Events that are structurally valid but semantically broken (e.g. `properties = null`) land but are filtered by NULL-guards in the metric queries, so they don't distort numbers.
- **Spikes** — Pub/Sub sits between the service and BigQuery and buffers bursts; the service publishes fast and returns, the subscription drains at its own pace.

3. API performance strategy — three tiers

Running a fresh full-scan per request won't survive load, so the API answers the same question three ways, each cheaper than the last. Measured on the deployed service ("promptly revenue per day"; bytes via BigQuery dry-run, latency includes network round-trip to europe-west1):

Tier	Source	Bytes scanned	Latency (cold)
1 — raw events (dedup + JSON extraction over ~40,778 rows)	`analytics.events`	~3 MB	~0.9 s
2 — pre-aggregated summary (~90 rows, one per app/day)	`analytics.daily_metrics`	~2.4 KB	~0.75 s
3 — in-memory cache hit	server RAM	0 B	~0.12 s

Bytes scanned is where the tiers separate sharply (~3 MB → ~2.4 KB → 0); the cold latencies for Tier 1 and Tier 2 are closer because both include the fixed network round-trip to europe-west1 (BigQuery bills on bytes, not wall-clock). The decisive latency win is the cache tier.

Measures taken, and what each trades away:

- **Pre-aggregation** into `daily_metrics` (one row per app/day). The heavy dedup + JSON work runs once per daily refresh, not per request. Tier 2 scans **~1,270× fewer bytes than** Tier 1 (BigQuery bills per byte, so this is the cost and latency win too). Trade-off: a summary can go stale between refreshes, and distinct-count metrics (`active_users`) aren't summable across days, so those fall back to the raw table. Refresh is a `CREATE OR REPLACE TABLE … AS SELECT` exposed at `POST /admin/refresh`; in production it's a scheduled query.
- **In-memory cache** (short TTL) — repeated questions serve from RAM (**0 bytes, ~5–7× faster**). This is what protects the Monday-9am moment when hundreds of dashboards ask the same questions. Trade-off: per-instance and briefly stale within the TTL; at larger scale it moves to a shared cache (Redis).

Why it won't slow as history grows: the summary is one row per (app, day) — a month is ~30 rows no matter how many billions of raw events accumulate, so serving stays flat.

4. The numbers behind every claim (generated → landed → response times)

Generated → landed (full run, replayed through the LIVE ingestion endpoint):

```
events generated:        40,887   (events.jsonl, seed=42)
rejected by service (400):  209    (109 invalid JSON + 100 missing a required field)
accepted (202):          40,678
rows landed in BigQuery: 40,580    (419 duplicate event_ids present in landed data)
unique after dedup:      40,161
```

The ~98-row gap between accepted and landed is malformed-but-structurally-valid events that passed the field check yet failed the BigQuery write (the dead-letter case) — the counts are honest and no bad data silently corrupted the table.

Validation: every metric was recomputed independently in Python from `events.jsonl` and diffed row-by-row against BigQuery. Money- and identity-based metrics (revenue, ARPU, conversion, retention) match **exactly**;

raw count metrics differ by ~11 rows (0.03%), fully explained by the malformed-data rejection + NULL-guards above. So the pipeline's numbers are trustworthy.

Response times (live, incl. network): raw-scan query ~0.9 s cold; cached response ~0.12 s (≈5–7× faster, 0 bytes). The heavy retention query (raw table, can't use the summary) is ~0.76 s cold → ~0.17 s cached.

5. User-lifecycle endpoints

The API is organized around install → engage → pay → churn: `/metrics/installs` (install), `/metrics/active_users` + `/metrics/feature_uses` + `/metrics/llm_tokens` (engage/backend), `/metrics/revenue` (pay), `/metrics/retention?day_n=N` (churn), and `/metrics/compare` for the leadership cross-app table. Each returns per-app, per-day series; `compare` returns one row per app. Every response reports `cached` and `bytes_processed` so query cost is visible.